

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	COMPUTACIÓN GRÁFICA, VISIÓN COMPUTACIONAL Y MULTIMEDIA (E)				
TÍTULO DE LA PRÁCTICA:	<i>CONTROLADORES DE ENTRADA DE PRIMERA Y TERCERA PERSONA</i>				
NÚMERO DE PRÁCTICA:	<i>04</i>	AÑO LECTIVO:	<i>2026A</i>	NRO. SEMESTRE:	<i>IX</i>
FECHA DE PRESENTACIÓN	<i>10/05/26</i>	HORA DE PRESENTACIÓN	<i>16:00</i>		
INTEGRANTE (s): Venero Guevara Christian Henry				NOTA:	
DOCENTE(s): <i>MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.</i>					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS 1. Introducción a. Cámara Virtual En los videojuegos en 3D, un sistema de cámara virtual tiene como objetivo controlar una cámara o un conjunto de cámaras para mostrar una vista de un mundo virtual tridimensional. b. Cámara en Primera Persona La cámara en primera persona es una vista empleada en los videojuegos en la cual el mundo se observa desde la perspectiva del personaje protagonista. La principal ventaja de esta vista es que proporciona un mayor realismo y sensación de presencia en el videojuego. La desventaja es que, al ver únicamente hacia el frente, el jugador debe rotar para conocer su entorno. c. Cámara en Tercera Persona La cámara en tercera persona muestra al personaje desde una perspectiva externa, generalmente ubicada detrás y por encima del personaje. Este tipo de cámara permite al jugador ver tanto al personaje como el entorno, facilitando la navegación en espacios tridimensionales. El principal desafío técnico de este tipo de cámara es evitar las colisiones con la geometría del entorno, lo cual puede causar que la cámara atravesase paredes u objetos, perdiendo la visibilidad del personaje.

d. Character Controller

El componente Character Controller de Unity permite controlar el movimiento de un personaje sin utilizar el sistema de físicas convencional basado en Rigidbody, lo cual brinda mayor control sobre el movimiento y la detección de colisiones del personaje.

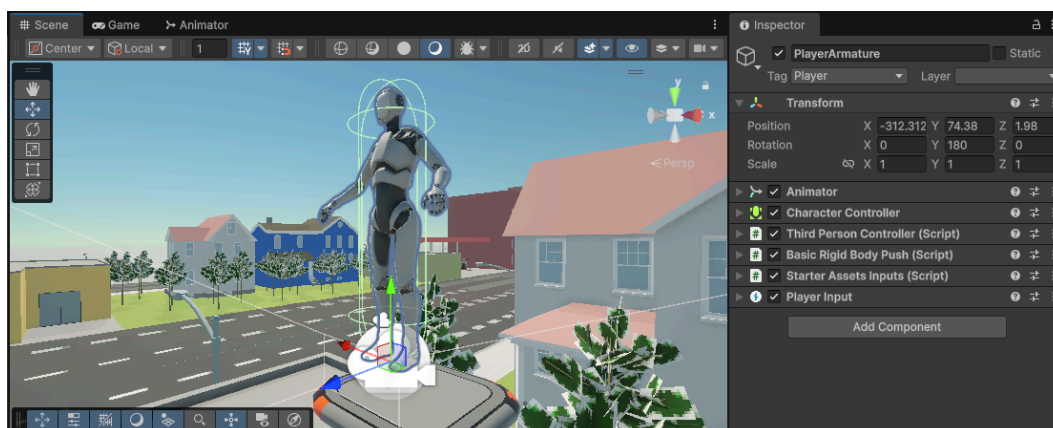
2. Controlador de Personaje y Cámara en Tercera Persona

a. Creación del Controlador de Personaje

Para el desarrollo del controlador de personaje se utilizó el paquete Starter Assets de Unity, el cual incluye un controlador base con animaciones y sistema de entrada. A continuación se describen los pasos realizados:

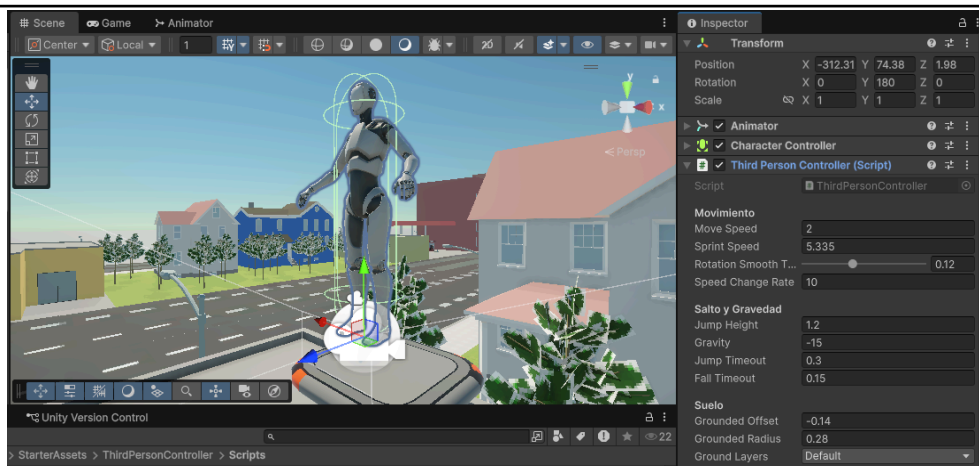
i. Importación del personaje:

1. Se importó el modelo PlayerArmature con su Animator Controller.
2. Se asignó el Tag "Player" al objeto PlayerArmature para su correcta identificación en los scripts de detección.



ii. Configuración del movimiento:

1. El script ThirdPersonController gestiona el movimiento del personaje con soporte para caminar, correr y saltar.
2. Se ajustaron los parámetros de velocidad: MoveSpeed = 2, SprintSpeed = 5.335, JumpHeight = 1.2 y Gravity = -15.

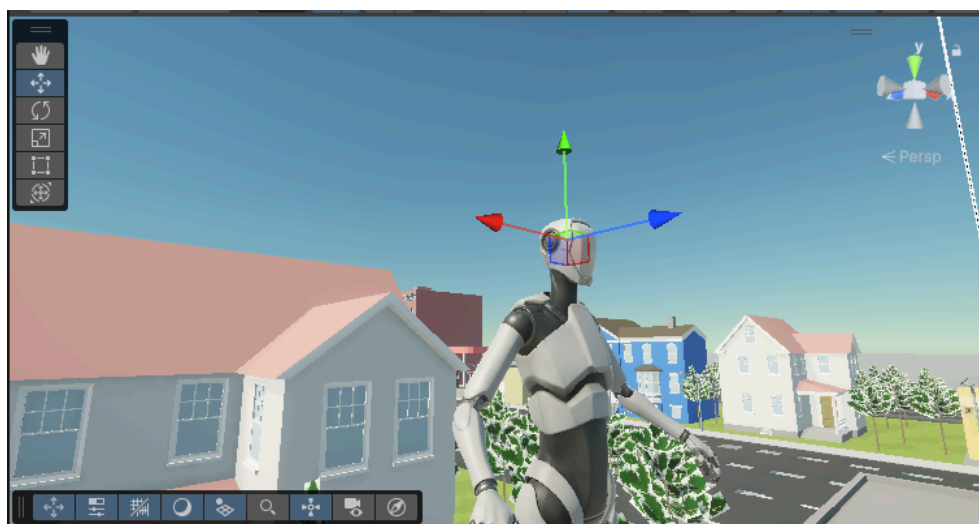


b. Sistema de Cámara en Tercera Persona

Para implementar el sistema de cámara se desarrolló un script personalizado denominado MultiPerspectiveCamera, el cual permite tanto la cámara en tercera persona como en primera persona, con cambio entre ambas mediante la tecla Q.

i. Configuración de los objetos objetivo de cámara:

1. Se creó un objeto vacío tpTarget como hijo del personaje, ubicado detrás y arriba del mismo para la vista en tercera persona.
2. Se creó un objeto vacío fpTarget como hijo del personaje, ubicado a la altura de los ojos para la vista en primera persona.



c. Solución de Colisiones de Cámara

El problema principal de las cámaras en tercera persona es que pueden atravesar la geometría del escenario, perdiendo la visibilidad del personaje. Para solucionar esto se implementó un sistema de detección de colisiones basado en Raycasts.

El método `GetCameraCollisionPoints` calcula cuatro puntos en las esquinas del plano cercano de la cámara (near plane). Desde cada uno de estos puntos se lanza un Raycast en la dirección de la cámara al personaje. Si alguno de estos rayos impacta con una geometría, la cámara se desplaza hacia adelante para evitar la oclusión:

CSharp

```
private Vector3[] GetCameraCollisionPoints(Vector3 direction)
{
    Vector3 center = follow.position + direction * (camera.nearClipPlane + 0.4f);
    Vector3 right = transform.right * nearPlaneSize.x;
    Vector3 up = transform.up * nearPlaneSize.y;
    return new Vector3[] {
        center - right + up, center + right + up,
        center - right - up, center + right - up
    };
}
```

En el método `LateUpdate` se itera sobre los cuatro puntos y se calcula la distancia mínima segura entre la cámara y el personaje, garantizando que la cámara nunca atravesase ninguna pared del escenario.

CSharp

```
void LateUpdate()
{
    Vector3 direction = new Vector3(
        Mathf.Cos(angle.x) * Mathf.Cos(angle.y),
        -Mathf.Sin(angle.y),
        -Mathf.Sin(angle.x) * Mathf.Cos(angle.y)
    );

    RaycastHit hit;
    float distance = defaultDistance;
    Vector3[] points = GetCameraCollisionPoints(direction);

    foreach (Vector3 point in points)
    {
        if (Physics.Raycast(point, direction, out hit, defaultDistance))
        {
            distance = Mathf.Min((hit.point - follow.position).magnitude, distance);
        }
    }

    transform.position = follow.position + direction * distance;
    transform.rotation = Quaternion.LookRotation(follow.position - transform.position);
}
```



Cámara en tercera persona.



Cámara en primera persona.

d. Raycast

Un Raycast en Unity es básicamente un láser invisible que se lanza desde un punto de origen en una dirección específica para detectar si choca con algo en el mundo del juego.

i. El concepto de "Puntos de Colisión" (GetCameraCollisionPoints)

Antes de lanzar el rayo, el código no sólo mira desde el centro de la cámara, sino que obtiene varios puntos (normalmente las esquinas del near plane).

Si solo lanza un rayo desde el centro, la esquina de la cámara podría atravesar una pared antes de que el centro la toque. Al usar múltiples puntos, se asegura que toda la "caja" de la cámara esté libre de obstáculos.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

ii. El bucle de detección (foreach y Physics.Raycast)

CSharp

```
foreach (Vector3 point in points)
{
    if (Physics.Raycast(point, direction, out hit, defaultDistance))
    {
        distance = Mathf.Min((hit.point - follow.position).magnitude,
        distance);
    }
}
```

1. Physics.Raycast: Lanza un rayo invisible desde cada punto hacia la dirección donde debería estar la cámara.
2. out hit: Si el rayo choca con algo (una pared, un edificio), guarda toda la información del choque en la variable hit.
3. Mathf.Min: Si varios rayos chocan con diferentes cosas, el sistema elige la distancia más corta. Esto garantiza que la cámara se acerque al personaje lo suficiente para no quedar "dentro" de ninguna pared.

iii. Posicionamiento Final

CSharp

```
transform.position = follow.position + direction * distance;
```

En lugar de colocar la cámara a la distancia máxima (defaultDistance), la coloca a la distance que calculó el Raycast. Si no hubo choques, será la distancia normal; si hubo un choque, la cámara se "pegará" al personaje para evitar el obstáculo.

iv. Orientación (Quaternion.LookRotation)

CSharp

```
transform.rotation = Quaternion.LookRotation(follow.position -
transform.position);
```

Una vez que la cámara se movió, esta línea asegura que siempre esté mirando hacia el objetivo (follow.position). Esto evita que la cámara pierda el encuadre del jugador mientras se ajusta por las colisiones.

3. Ciudad con Parkour y Múltiples Cámaras

a. Descripción de la Temática

La escena consiste en un entorno urbano de ciudad en el cual el jugador debe recorrer un circuito de parkour con obstáculos, plataformas y trampas. El objetivo del juego es llegar a la plataforma final sin caer al suelo.



- i. Temática: Ciudad urbana con elementos de parkour.
- ii. Objetivo: Llegar a la plataforma de victoria esquivando obstáculos.
- iii. Tipos de cámara utilizados: Tercera persona (exploración y parkour) y primera persona (modo inmersivo).
- iv. Cambio de cámara: Mediante la tecla Q el jugador alterna entre ambas perspectivas.



	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

b. Sistema de Cambio de Cámaras

Se implementó un sistema dual de cámaras integrado en el script MultiPerspectiveCamera que permite alternar entre la vista en tercera persona y la vista en primera persona durante el gameplay:

i. Cámara en Tercera Persona:

1. Utilizada durante las secciones de exploración y saltos de plataforma.
2. Permite visualizar al personaje completo y el entorno circundante.
3. Incluye zoom con la rueda del mouse (distancia entre 2 y 7 unidades).
4. Protección contra colisiones mediante sistema de Raycasts.

ii. Cámara en Primera Persona:

1. Activada al presionar la tecla Q, ideal para secciones más inmersivas.
2. El mesh del personaje se oculta automáticamente al cambiar a esta vista.
3. Se reactiva el mesh del personaje al volver a la tercera persona.

El cambio entre cámaras se gestiona mediante el método ChangePerspective, el cual actualiza el objeto objetivo que sigue la cámara y controla la visibilidad del mesh del personaje según la perspectiva activa.

CSharp

```
void ChangePerspective(bool ThirdPerson)
{
    if (ThirdPerson)
    {
        follow = tpTarget;
        if (disablePlayerMesh)
            playerMesh.SetActive(true);
        tPerson = true;
    }
    else
    {
        follow = fpTarget;
        if (disablePlayerMesh)
            playerMesh.SetActive(false);
        tPerson = false;
    }
}
```

Finalmente, la integración del script MultiPerspectiveCamera con el ThirdPersonController permitió consolidar un sistema de navegación adaptable a las necesidades del entorno urbano de la escena. Mediante la implementación del sistema de colisiones por Raycast y la gestión dinámica del playerMesh, se garantizó una transición fluida entre la perspectiva de primera y tercera persona sin comprometer la inmersión visual ni la estabilidad física del astronauta.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

II. SOLUCIÓN DEL CUESTIONARIO

1. ¿Qué problemas pueden presentar las cámaras de tercera persona?

- Colisiones: La cámara puede atravesar paredes, techos u objetos del escenario, perdiendo la visibilidad del personaje. Este es el problema más frecuente.
- Oclusión del personaje: Objetos del entorno pueden interponerse entre la cámara y el personaje, bloqueando la visión del jugador.
- Comportamiento errático en espacios reducidos: En pasillos o espacios pequeños la cámara puede tener movimientos bruscos e impredecibles al intentar encontrar una posición válida.
- Gimbal lock: En ciertas orientaciones extremas el sistema de rotación puede perder un grado de libertad, causando rotaciones incorrectas.

2. ¿Cómo han realizado el control de múltiples cámaras en la escena?

El control de múltiples cámaras se implementó mediante un único script MultiPerspectiveCamera que gestiona ambas perspectivas (tercera y primera persona) a través de los siguientes mecanismos:

- Objetos objetivo (targets): Se definieron dos Transform de referencia, tpTarget para la cámara en tercera persona y fpTarget para la primera persona, ambos como hijos del personaje.
- Variable de estado tPerson: Un booleano que indica la perspectiva activa y determina qué target sigue la cámara y si el mesh del personaje es visible.
- Cambio por input: Al presionar la tecla Q se invoca el método ChangePerspective, el cual actualiza el target activo y la visibilidad del mesh.
- Control del cursor: El GameManager gestiona el estado del cursor (bloqueado durante el gameplay, libre en los menús) en coordinación con la cámara para que el cursor no interfiera con el control de la cámara durante el juego.
- Pausa por Time.timeScale: La cámara verifica Time.timeScale en el Update para desactivarse automáticamente cuando se muestra algún menú, permitiendo que el cursor quede libre para interactuar con los botones.

III. CONCLUSIONES

- Control de Oclusión con Raycast: Los Raycasts son esenciales en cámaras de tercera persona para detectar paredes; sin ellos, la cámara atravesaría los edificios de la ciudad y el jugador perdería de vista al personaje.
- Detección en el Near Plane: Lanzar rayos desde las cuatro esquinas del plano cercano de la cámara (y no solo desde el centro) ofrece una detección de colisiones mucho más exacta en pasillos estrechos.
- Versatilidad de Perspectivas: Implementar un sistema dual permite que el juego sea más inmersivo, alternando entre la exploración (tercera persona) y la precisión (primera persona) de manera fluida.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

- Gestión de la Visibilidad del Jugador: Al cambiar a primera persona, el script desactiva automáticamente el playerMesh. Esto es esencial para evitar que la cámara "vea" el interior de la cabeza del modelo 3D mientras el jugador controla al personaje.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Unity Technologies, "Character Controller," *Unity Real-Time Development Platform Manual*, 2024. [En línea]. Disponible en: <https://docs.unity3d.com/Manual/class-CharacterController.html>. [Accedido: 7-may-2026].
- [2] Unity Technologies, "Input System Package," *Unity Documentation*, 2024. [En línea]. Disponible en: <https://docs.unity3d.com/Packages/com.unity.inputsystem@1.7/manual/index.html>. [Accedido: 7-may-2026].
- [3] Microsoft Learn, "Raycasting in Unity," *Game Development Documentation*, 2023. [En línea]. Disponible en: <https://learn.microsoft.com/en-us/training/modules/create-simple-game-unity-input-system/>. [Accedido: 7-may-2026].
- [4] G. Williams, "Third Person Camera Systems and Collision Detection in 3D Environments," *Gamasutra (Game Developer)*, 2022. [En línea]. Disponible en: <https://www.gamedeveloper.com/design/third-person-camera-navigation>. [Accedido: 7-may-2026].
- [5] Unity Technologies, "Camera.nearClipPlane," *Unity Scripting Reference*, 2024. [En línea]. Disponible en: <https://docs.unity3d.com/ScriptReference/Camera-nearClipPlane.html>. [Accedido: 7-may-2026].
- [6] Mixamo (Adobe), "Character Rigging and Humanoid Animation for Unity," 2024. [En línea]. Disponible en: <https://helpx.adobe.com/creative-cloud/help/mixamo-rigging-animation.html>. [Accedido: 8-may-2026].
- [7] Unity Technologies, "Mathf.SmoothDampAngle," *Unity Scripting API*, 2024. [En línea]. Disponible en: <https://docs.unity3d.com/ScriptReference/Mathf.SmoothDampAngle.html>. [Accedido: 8-may-2026].
- [8] NVIDIA Developer, "Efficient Raycasting for Real-Time Physics," *NVIDIA Technical Blog*, 2023. [En línea]. Disponible en: <https://developer.nvidia.com/blog/real-time-physics-simulation-for-games/>. [Accedido: 9-may-2026].
- [9] Starter Assets Team, "Third Person Controller - Starter Assets Documentation," *Unity Asset Store*, 2024. [En línea]. Disponible en: <https://assetstore.unity.com/packages/essentials/starter-assets-third-person-character-controller-196526>. [Accedido: 10-may-2026].